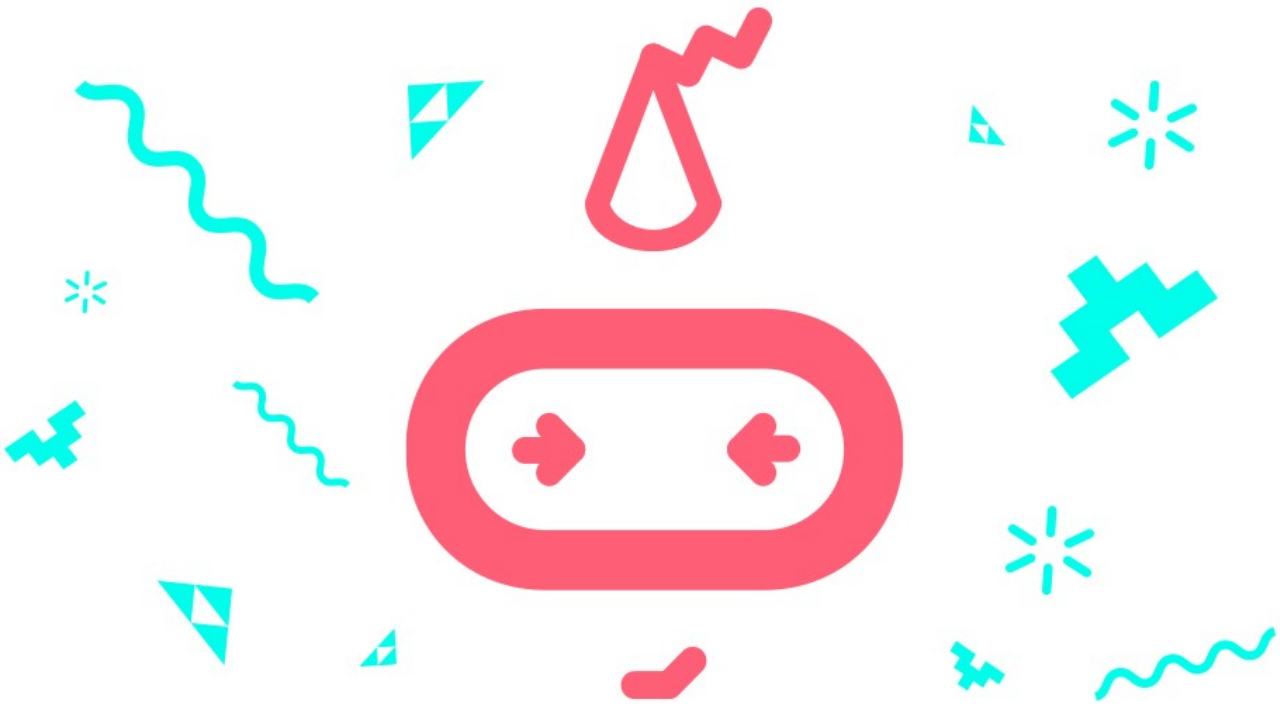
Introduction to Computer Science

Unit 1:
Making with micro:bit

Student workbook
makecode.microbit.org

Table of Contents



Welcome

Welcome to Computer Science with MakeCode and micro:bit! We are excited for all that you will discover and learn while inventing and creating with the micro:bit. Be prepared to grow and build your skills. Good luck and have fun!

Overview

Unit summary

This unit introduces the micro:bit and explains how it can be used. The focus is on incorporating the physical micro:bit into a basic making activity. You will be introduced to the MakeCode programing environment and learn the process for importing a program and downloading it onto the micro:bit. In the unplugged activity, you will learn about design thinking and prototyping by interviewing each other about an ideal imaginary pet. In the final project, you will use what you discovered in your interviewing and prototyping to craft a micro:pet to your partner's specifications with the micro:bit.

Learning goals

During this unit, you will:

- Exercise creativity, engineering, and resourcefulness by coming up with ideas for using simple household or classroom materials to accommodate the micro:bit's size and weight as part of your micro:pet project.
- Test and iterate using different materials and sizes in order to create an optimal design to house the micro:bit and battery pack.
- Become familiar with the MakeCode programming environment.
- Learn how to download programs from the computer to the micro:bit.
- Exercise communication and collaboration and apply the design thinking process to develop an understanding of a problem or user need and iteratively design an optimal solution.
- Apply your understanding of the problem in a creative way by making a "micro:pet" creature for your partner.

Lesson A: The micro:bit is for making

Overview: Making with micro:bit

Who likes to make and create things? Did you know that coding (or computer programming) is all about making, designing, and creating? All computer programs are created to solve a problem or serve a purpose. The problem may be local or global, and the purpose may be anything from helping doctors treat patients to pure entertainment. A program, which can be called code or sometimes a script, tells a computer what to do. A program can be instructions for any kind of computer—like a laptop, smart phone, robot, and more—to accomplish something useful for people and society.

In this course you're going to learn about coding and computer science by making different projects. You'll see what knowing how to code allows you to build and create! It's all about making, building, crafting, and construction.

You'll use different supplies and materials, but the two main tools are:

- A micro:bit, which is a microcontroller board. Think of it as a type of small computer. It can be used for all sorts of cool creations from robots to musical instruments—the possibilities are endless.
- Microsoft MakeCode is an easy and powerful coding environment that allows you to give the micro:bit instructions. Just like humans speak different languages, computers do too. In MakeCode, you can code in blocks or JavaScript. It's easiest to learn to code with the block-based language, so we'll use that in the course, but feel free to experiment with JavaScript.

Use this space to interview your partner

1. Do you have a pet?

2. If yes, what is it? What do you like about your pet? What do you dislike?

3. If you don't have a pet, what kind of pet would you like to have? Why?

4. Is there anything you wish your pet could do? Why?

5. What kind of activities would you do with your pet?

6. How would you care for it? Where would it live?

7. Tell me about your ideal pet.

Lesson B: Introduction to MakeCode

Activity: Introduction to MakeCode

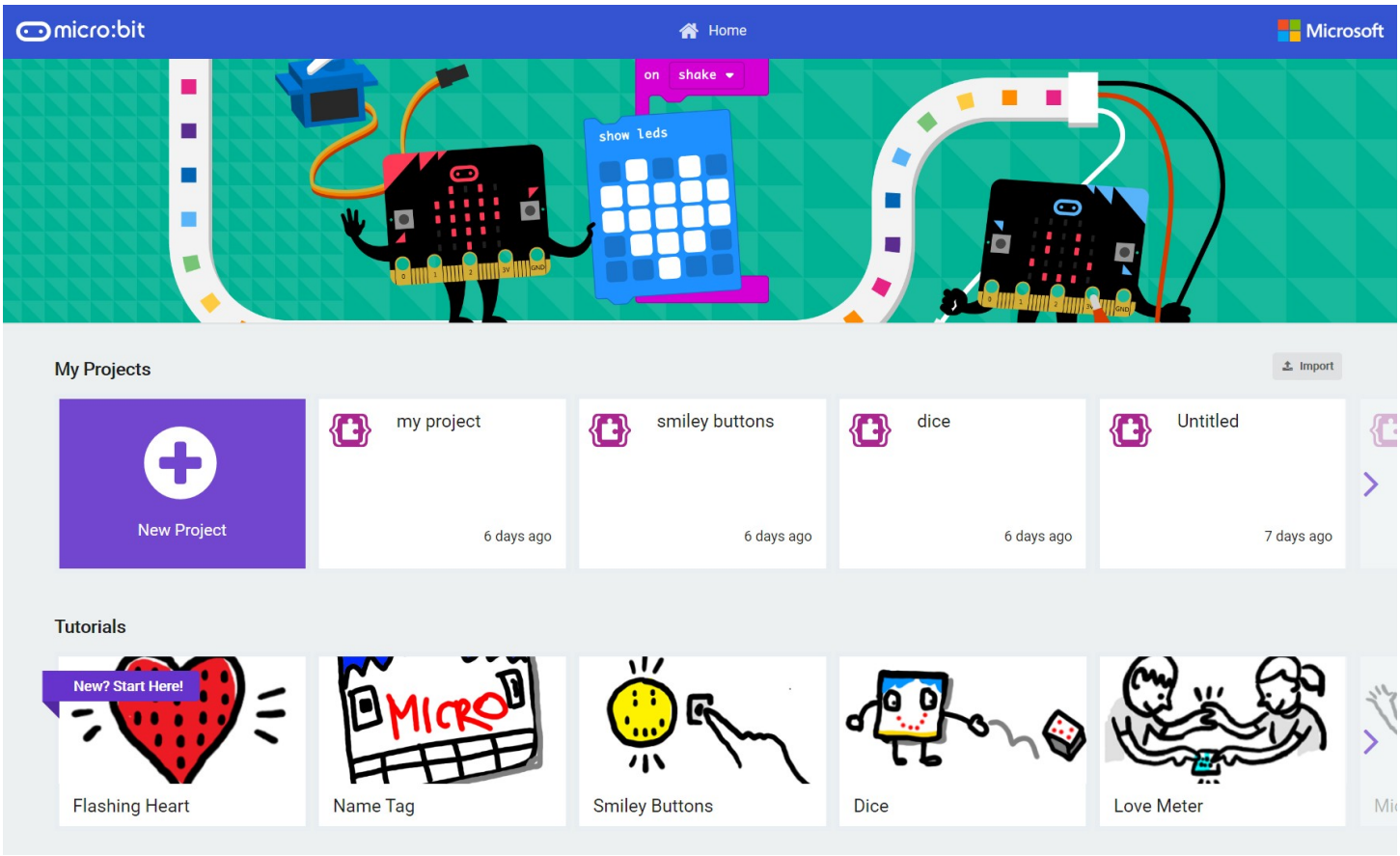
Overview

You will import a simple program in Microsoft MakeCode and download it to your micro:bit using a USB cable.

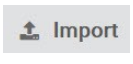
Open Microsoft MakeCode and upload a program

On your computer, laptop, or tablet, open a browser window to makecode.microbit.org.

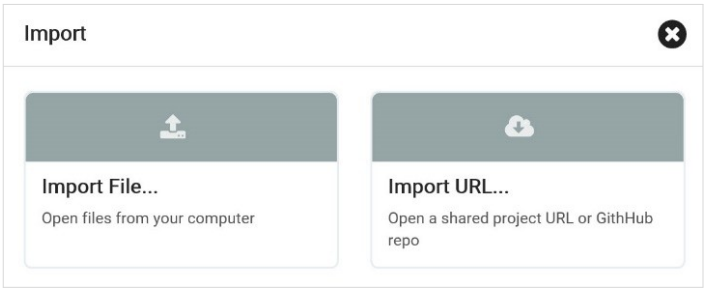
This is the MakeCode home page:



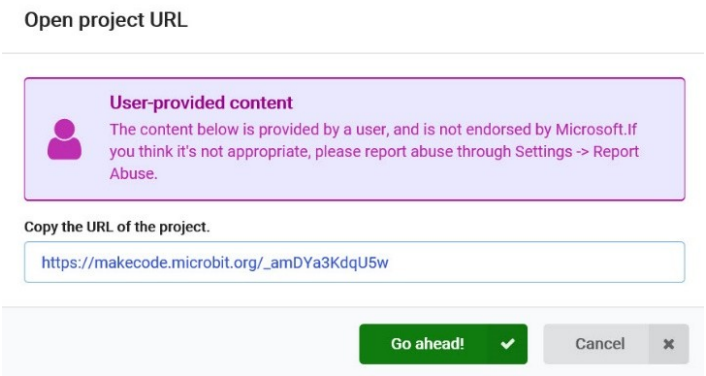
The My Projects area lets you start a new project or select an existing project. Or, you can import a program, which we'll do now.

Select the Import button  on the right side of the screen, under the banner.

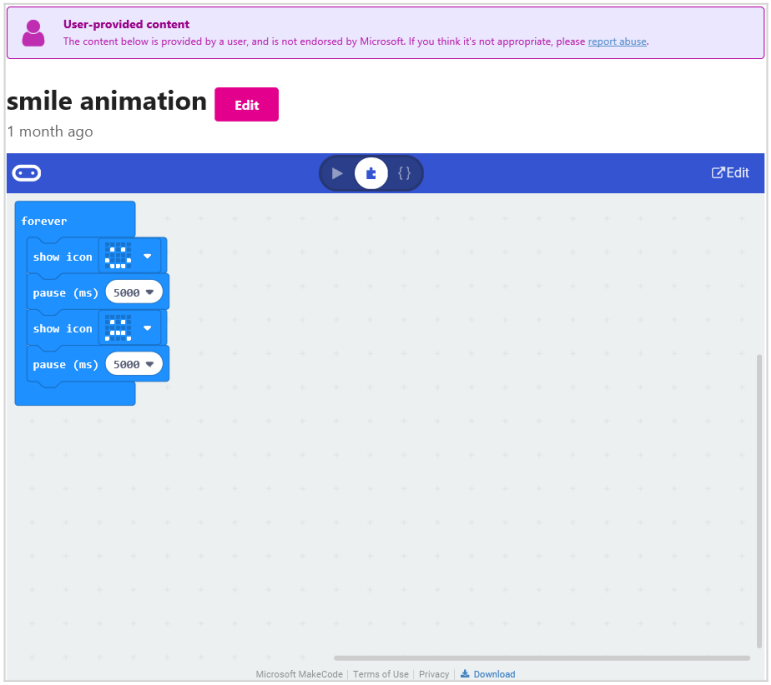
In the Import window, select Import URL…



Copy or type the smile animation project URL in the field: https://makecode.microbit.org/_amDYa3KdqU5w and select Go ahead! The link is case sensitive, and you need to include the underscore.



This opens the imported project in a preview page. Select **Edit** to open it in the MakeCode editor.

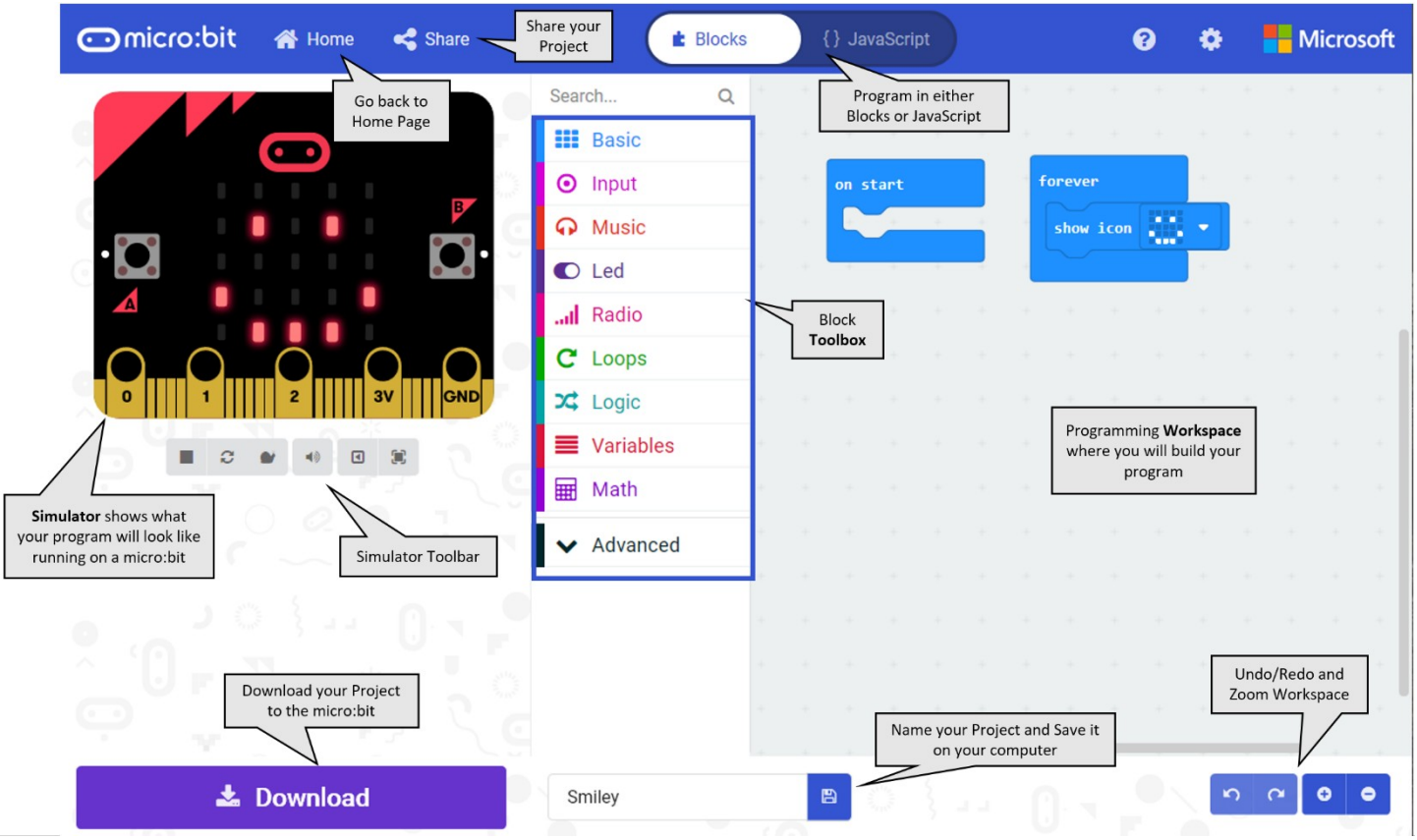


Tour of Microsoft MakeCode

There are three main areas of the MakeCode editor:

- Simulator: On the left side of the screen, you will see a virtual micro:bit that will show what your program will look like running on a micro:bit. This is helpful for debugging and instant feedback on program execution.
- Toolbox: In the middle of the screen, there are a number of different categories or Toolbox drawers, each containing a collection of different blocks that can be dragged into the Workspace on the right.
- Workspace: on the right side of the screen is the programming or coding Workspace where you will create your program. Programs are constructed by snapping blocks together in this area, like the program you just uploaded. The color of the blocks identifies their category. All of the blocks that make up the program above come from the Basic Toolbox category, which is light blue.

Explore each of the call outs:

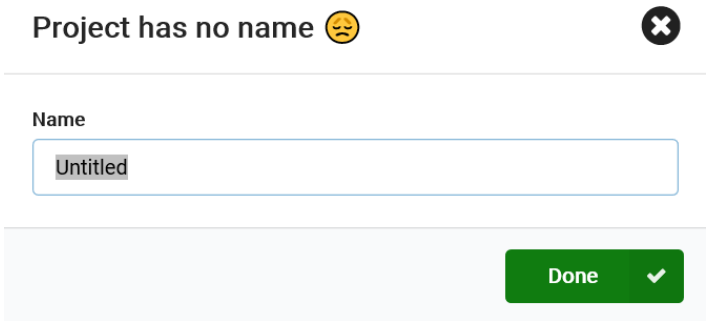


Name, save, and publish a MakeCode project

Name your MakeCode project

It's good coding practice to name your coding programs, which can be done a couple of ways:

- When you open a new project, the default project name is: "Untitled" and shows in the name field (along the bottom of the editor to the right of the purple Download button). Enter your desired name.
- Alternatively, you can name the program when you exit a project. Select the Home button, then enter the desired name, and select Done.



- When you've imported a program, it will default to the published name, i.e., Smiley in the above image. You can change the name as desired.

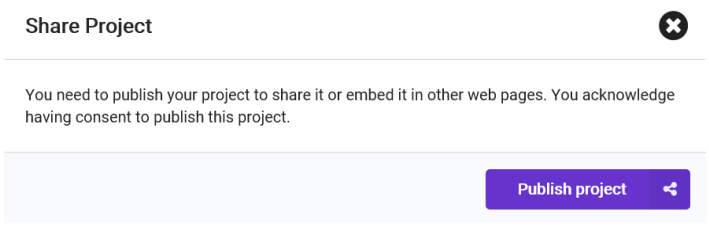
You will then see the project listed in the My Projects area of the home page to access at another time. A named or Untitled project is saved according to the login of whatever browser is being used. When you clear the cache of the browser, the projects will be lost unless you've saved them as a .hex file or published the project to get the URL.

Save a project as a .hex file

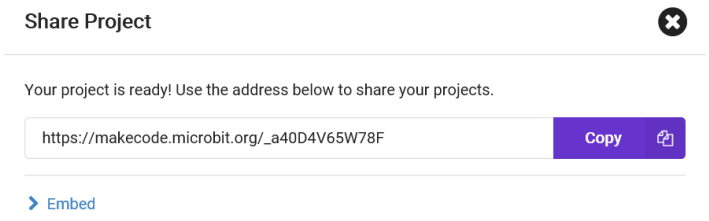
When you're in an open project and select the Save button, the program will download as a .hex file to your computer to the location your browser is set to save downloads. This .hex file can then be shared with others, who can import the .hex file into MakeCode.

Publish a project to get a sharing link

When you're in an open project, select the Share button (in the top task bar to the right of the Home button). In the Share Project window, select the purple Publish project button.



You can then copy the sharing link. Only people with that link will be able open the published version. There is also an option to get Embed code instead.



Important!

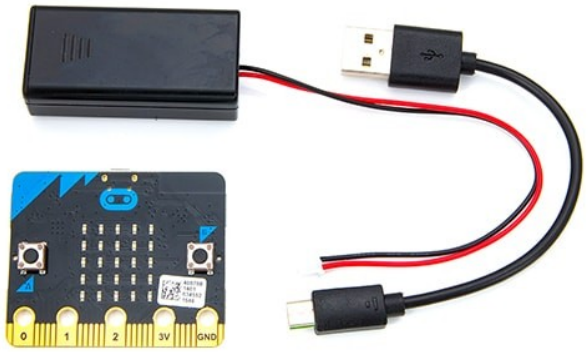
- The sharing link is case sensitive.
- Be sure to save or write down the link in a safe place as it's not searchable to find later.
- If you lose the sharing link, you can publish the project again from the MakeCode editor to get a new sharing link of the same program.

Write the link to your project here:

Download a MakeCode program to the micro:bit

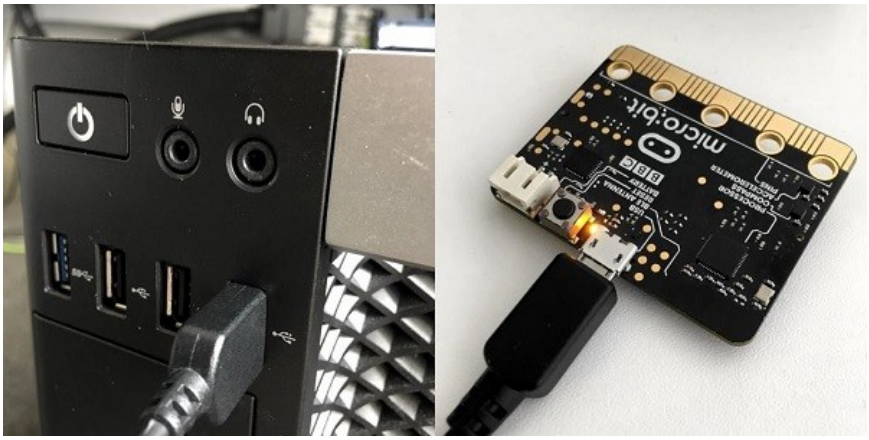
Note: It's time to discuss equipment safety procedures, including:

- Always ground/earth yourself to discharge any static electricity by touching a metal object briefly before touching the micro:bit.
- Always have dry hands and only touch the edges of the micro:bit.
- Don't place any metal objects across the printed circuits on the micro:bit. Doing so can cause a short circuit and damage the micro:bit.



To download the file to your micro:bit, you must connect it to your computer's USB port using a micro-USB cable. The micro:bit will draw power from your computer through the USB connection, or you can connect an optional battery pack so it can function even after it is unplugged from the computer. Once plugged in, the micro:bit shows up on your computer like a USB flash drive.

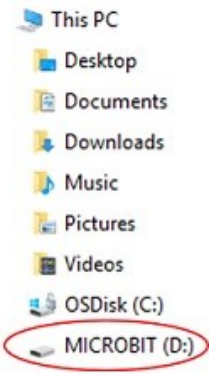
Use the micro-USB cable to connect the micro:bit to your device.



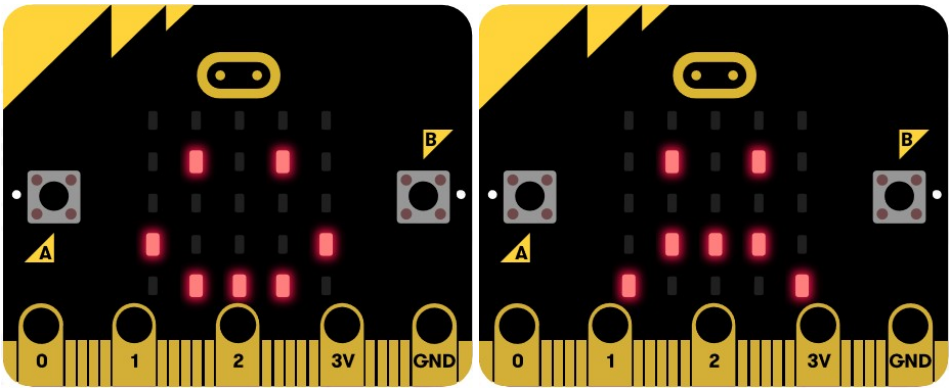
Select the purple Download button in the lower left of the MakeCode 

screen.

- Depending on your browser, the downloaded .hex file will either be in the Downloads folder, or the browser will prompt you to save the file to a specific location.
- If you're using the MakeCode for micro:bit Windows 10 app, the file will automatically copy to the micro:bit upon selecting the purple Download button.



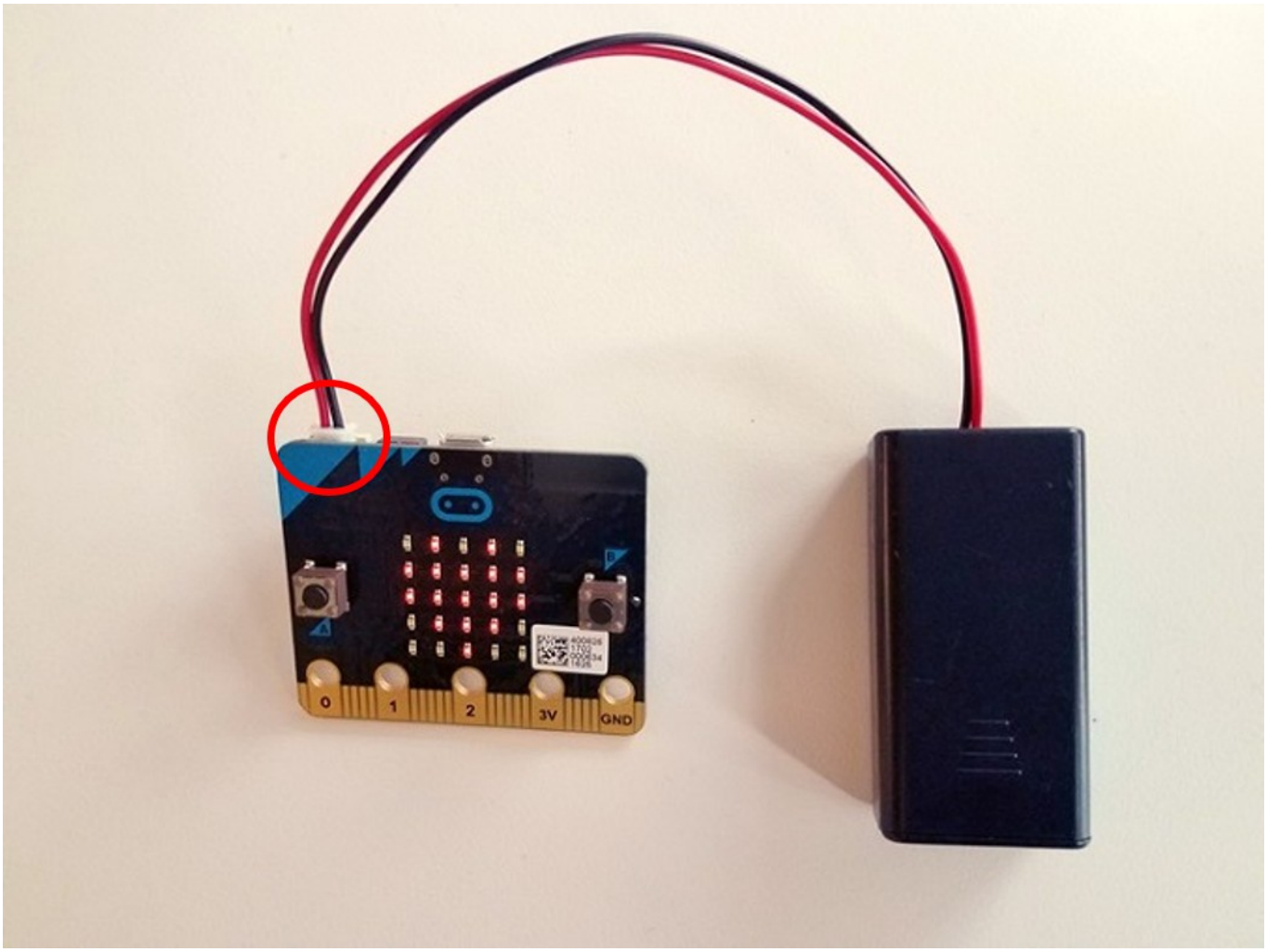
To move the program to your micro:bit, drag the downloaded .hex file to the MICROBIT drive, as if you were copying a file to a flash drive. The program will copy over, and it will begin running on the micro:bit immediately. It should look like this, showing a series of faces:



The micro:bit will hold one program at a time. It is not necessary to delete files off the micro:bit before you copy another onto the micro:bit; a new file will just replace the old one.

Connect the micro:bit to the battery pack

To prepare for the project in the next lesson, you'll attach the battery pack to the micro:bit using the white connector. That way you can build the micro:bit into your project design without having to connect it to the computer.



Lesson C: micro:pet project

This project is an opportunity to create a micro:pet for the partner you interviewed in the unplugged activity in Lesson A. Review your notes in your workbook, including the design statement and diagram prototypes you drew to remember what your partner finds appealing in a pet. Then, you'll use the crafting materials to create a prototype model of a pet your partner would like.

Continue with the design thinking process to build a micro:pet that:

- Matches your partner's needs
- Supports the micro:bit and its battery pack
- Allows you to easily access the micro:bit to turn it on and off

Your design should use whatever materials are available to support the micro:bit so that its face is showing. You can be creative and decide how to mount the board and how to decorate your critter.

Reminder:

- Only use tape to attach the micro:bit to project crafting materials.
- Don't use glue or draw on the micro:bit itself.
- Don't place any metal objects across the printed circuits on the board, as this can cause a short circuit and damage the micro:bit.

Think about the following questions when you construct it:

- Will it be an animal? A plant? A robot? A bug?
- Will it have any moving parts?
- If it moves, how can you hold the micro:bit securely?

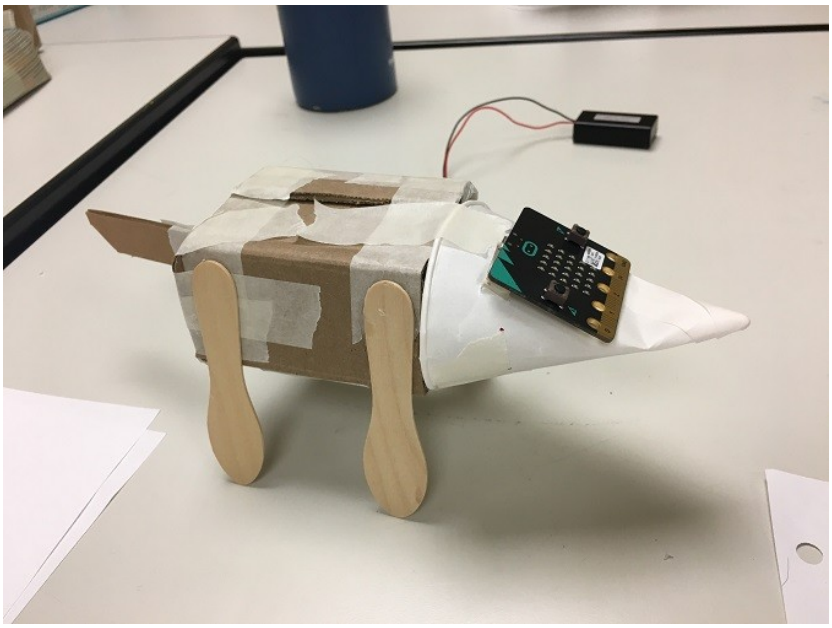
Remember the design thinking process:

1. Empathize by learning more about your target audience (your partner).
2. Define: Understand and identify your audience's problems or needs (what qualities of a pet are important for your partner).
3. Ideate: Brainstorm several possible creative solutions (ideas for different pets).
4. Prototype: Construct rough drafts or sketches of your ideas.
5. Test your prototype solutions and refine until you come up with the final version.

In Lesson A, you completed steps 1-4. Although you did some prototyping, you might want to sketch a few more designs on paper first, then test those ideas with your partner to see which aspects of those designs they find most appealing.

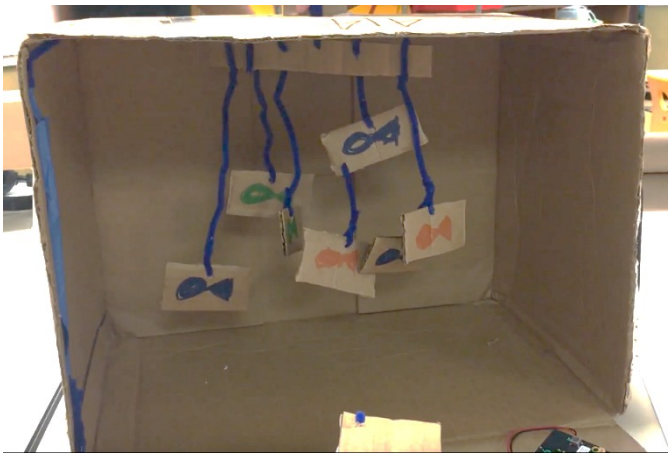
Project examples

Dog

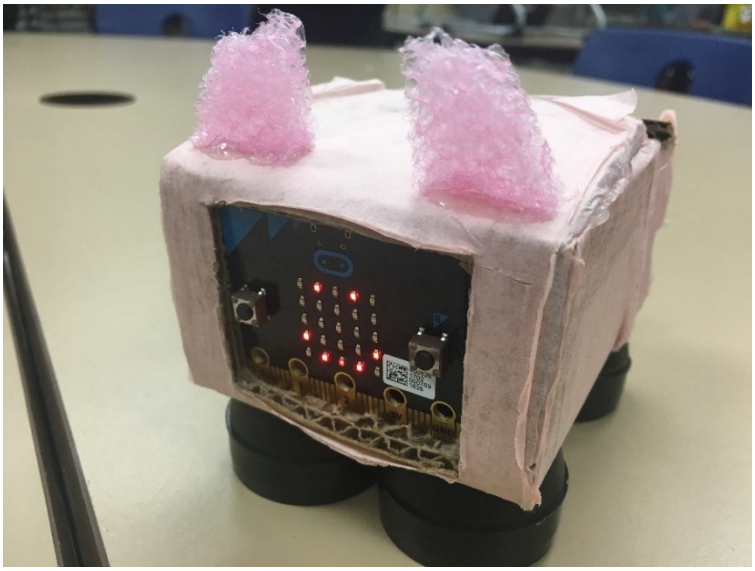


micro:pet Fish Tank

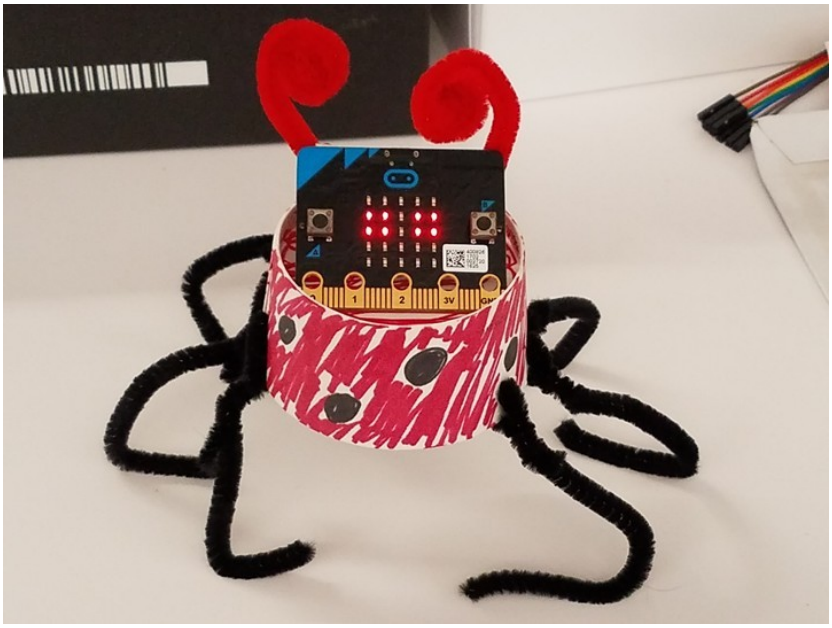
You can see this project in action at youtube.com/watch?v=2ZCDB-a_uRY (0:04).



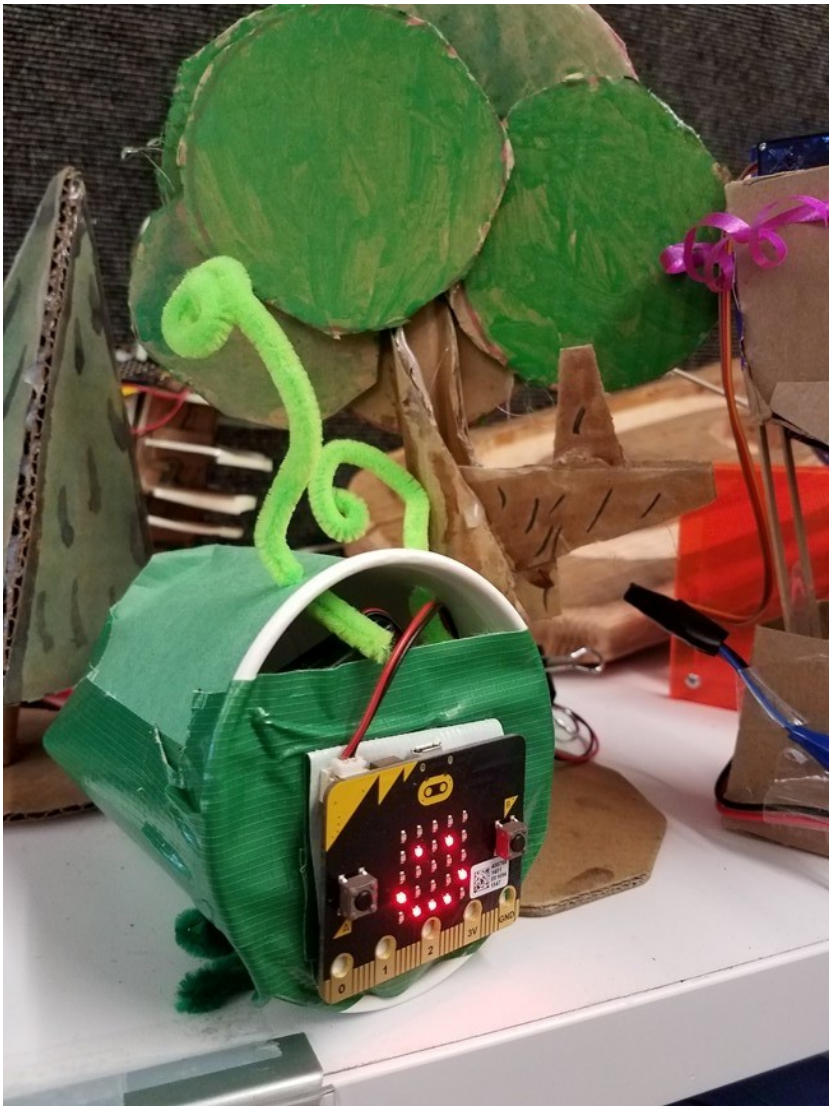
Pink Piggy



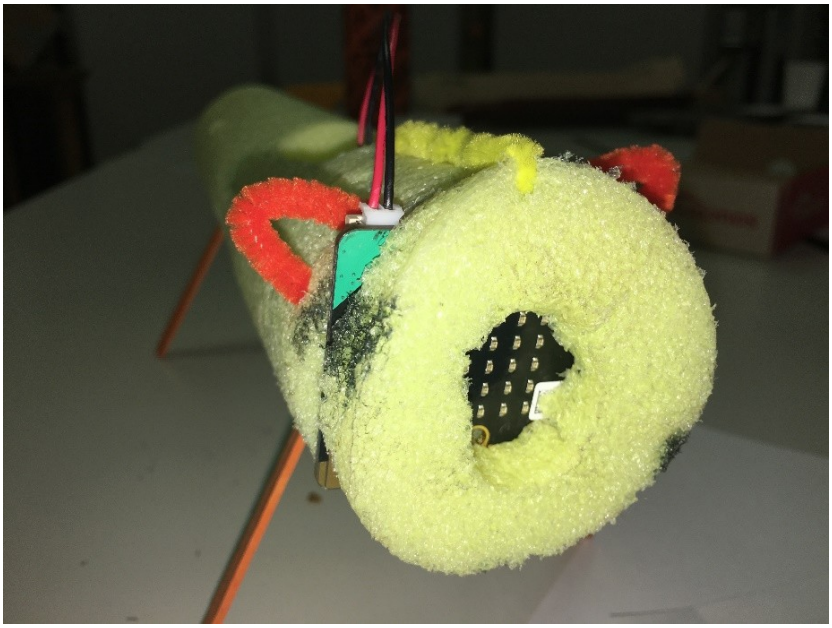
Ladybug



Caterpillar



Fox



Robot


Logitech
Wired Keyboard for iPad
Lightning™ Connector
Clavier filaire pour iPad
Connecteur Lightning

Ideas for mods

Mods, short for modifications, are examples of optional challenges you can do to make your project extra special.

- Find a way to make part of the animal move.
- Give your animal a natural habitat.
- Create a way to carry your animal.
- Create an animal that reacts when you pet it or move it (find a way to detect when the micro:bit is moved or when its position changes in a certain way.)

Project expectations

Follow the design thinking approach and make sure that the micro:pet meets the required specifications:

- The program downloads properly to the micro:bit
- The micro:bit is supported so the face is showing
- The micro:bit can be turned on and off without taking the critter apart
- Provide your notes from the interview process (or provide a picture)
- Provide the written Reflection Diary entry (which we'll talk about after you complete your project)

The design thinking process:

1. Empathize by learning more about your target audience (your partner).
2. Define: Understand and identify your audience's problems or needs (what qualities of a pet are important for your partner).
3. Ideate: Brainstorm several possible creative solutions (ideas for different pets).
4. Prototype: Construct rough drafts or sketches of your ideas.
5. Test your prototype solutions and refine until you come up with the final version.

Project scoring rubric

Assessment elements	1	2	3	4
Project	Project is missing four or more of the required elements.	Project is missing two or three of the required elements.	Project is missing one of the required elements.	Project addresses all required elements.

Use this space to track your design thinking process:

Reflection Diary

Expectations

Write a reflection of about 150–300 words addressing the following points:

- Summarize the feedback you got from your partner on your idea. How would you revise your design if you were to go back and create another version?
- What was it like to have someone designing a pet for you? Was it a pet you would have enjoyed? Why or why not? What advice did you give them that might help them redesign?
- What was it like to interview your partner? What was it like to be listened to?
- What was something that was surprising to you about the process of designing the micro:pet?
- Describe a difficult point in the process of designing the micro:pet and explain how you resolved it.
- Publish your MakeCode program and include the link.

Diary entry scoring rubric

Assessment elements	1	2	3	4
Diary entry	Diary entry is missing three or more of the required elements.	Diary entry is missing two of the required elements.	Diary entry is missing one of the required elements.	Diary entry addresses all elements.

Glossary of key terms

Block coding: A programming language found in coding editors—such as Microsoft MakeCode and Scratch—that uses different colored and shaped blocks that connect in a specific order to allow beginners to learn about coding concepts without having to worry about syntax. Also known as block programming.

Code or computer program: A set of instructions that a computer can follow. For example, an app or a game, like Minecraft, is a computer program. The terms can be used interchangeably.

JavaScript: A text-based programming language that uses letters, numbers, and symbols. It's one of the most popular programming languages in the world.

Microsoft MakeCode: A coding editor in Code Connection that lets you code with two programming languages: Block or JavaScript.

Prototype: A rough draft, sketch, or working model of your idea. The purpose of prototyping is to gather more feedback to help you in your final design.